

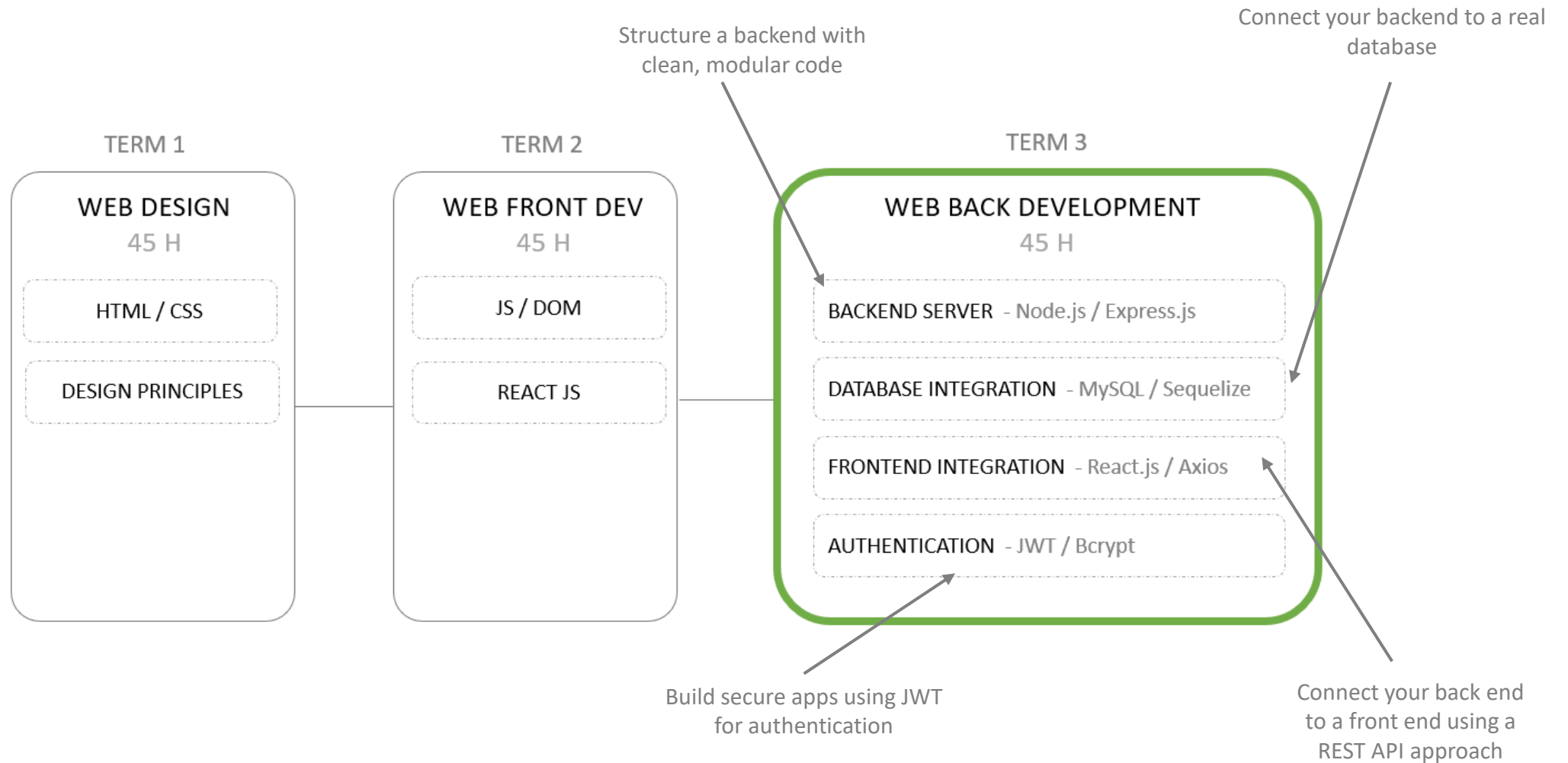
BACK-END DEVELOPMENT COURSE

WEEK 1 – KICK OFF !



COURSE SCOPE

Build full-stack web applications



COURSE AGENDA

Week	Objectives	EXAMS	PROJECTS
W 01	Intro to Node.js & Server Concepts		
W 02	Native HTTP Server & Manual Routing		
W 03	Express Basics + POST + Middleware		
W 04	REST API Design + Modular Express		
W 05	React + Axios Integration		
W 06	SQL with MySQL (Without ORM)		PROPOSAL
W 07	Sequelize Basics: Models & Relationships		
W 08	Sequelize Routes + Querying + Seeders		
W 09	Full Auth Flow (JWT + React + Sequelize)		
W 10	Project Presentations & Revisions		JURY
W 11	Final Exam	FINAL EXAM	

COURSE EVALUATION

TYPE OF EVALUATION	RATIO
Attendance & Participation	10 %
Assignments	20 %
Project	30 %
Final Exam	40 %

COURSE PROJECT

As part of this course, you will undertake a **project in groups of 3 (or 2)**

1

- **Define** your backend project
- **Present how you roadmap plan** to develop this backend in 4 weeks

2

- **Present your Project to the Jury**

Learning period



Week 1

Week 6

Week 10

BACK-END DEVELOPMENT

WEEK 1 – Server Client Architecture & Node.JS

CADT
IDT





Session objectives



- ✓ Understand the flow of a **client-server architecture**
- ✓ Describe the core **responsibilities of a server**
- ✓ Explain what **Node.js** is and why it is used
- ✓ Illustrate how the Node.js **event loop works**
- ✓ Differentiate between **synchronous and asynchronous** programming





What do you **already** know?

Discuss in teams on the following questions

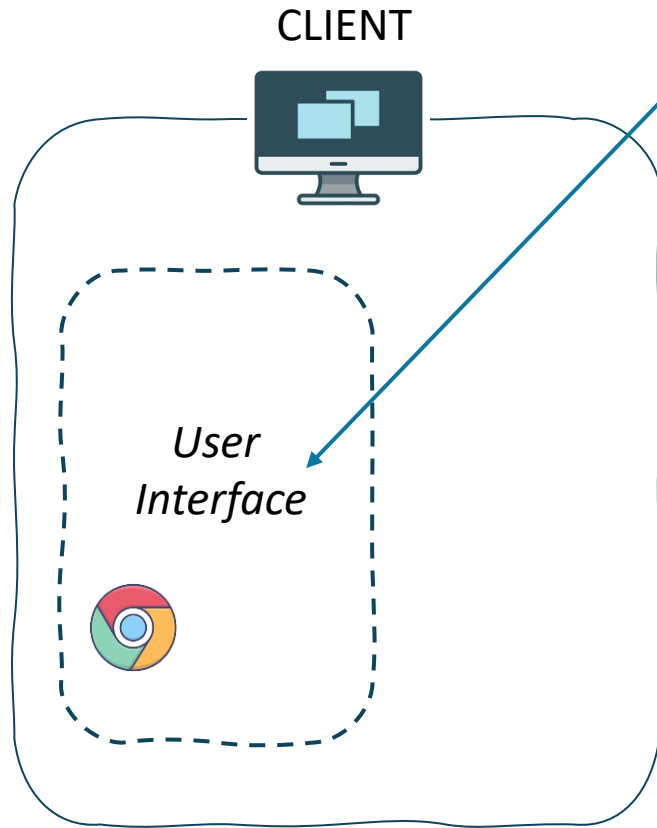
1. Where do you think **web application** store their data?
2. What do you think a **server** is?
3. What do you think a **client** is?
4. What's happen when **you open a website** in your **browser**?
5. What is a server **request** ? What is a server **response**?

The Client–Server Flow

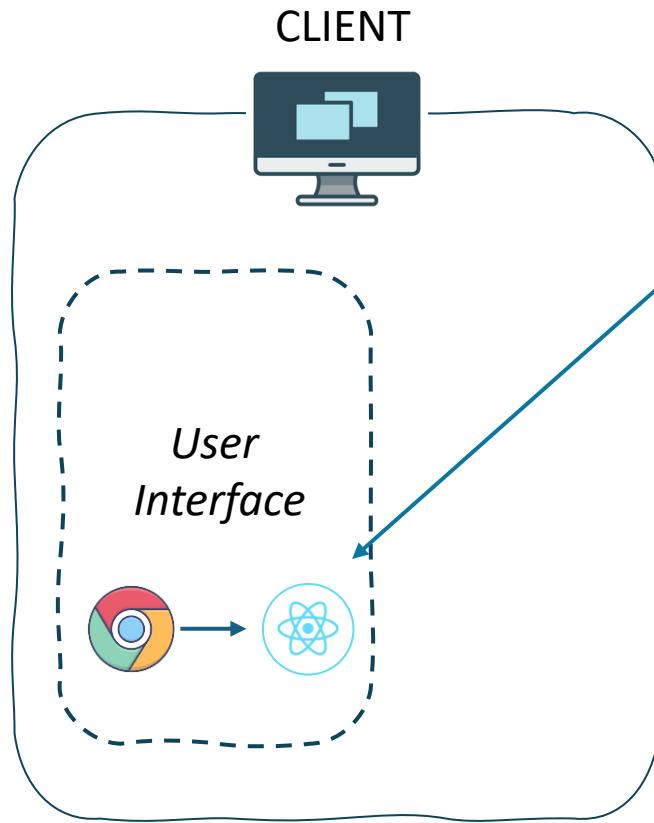
STEP 1

The User Clicks a Button on the Web Browser

Example: A button says "Get Profile Info"



The Client–Server Flow

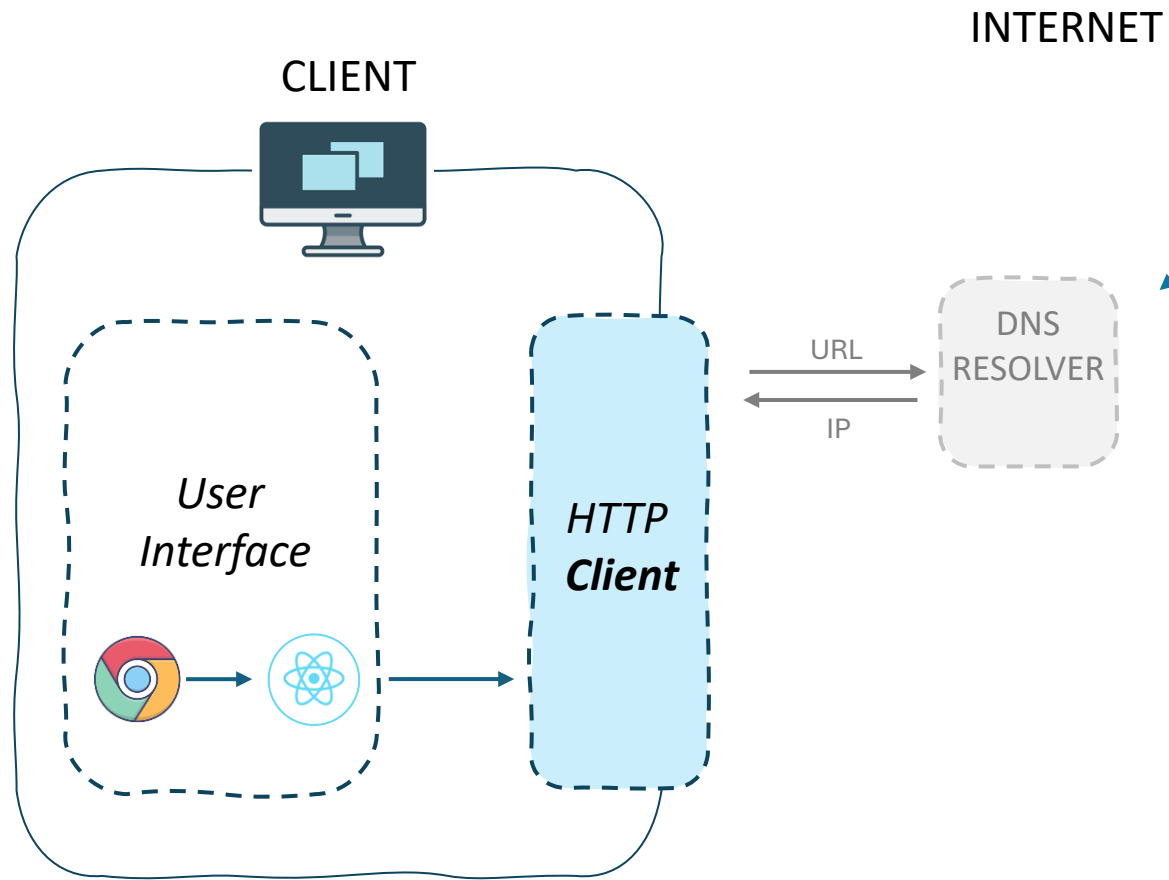


STEP 2

The client logic code will trigger an API call using `fetch()`, `axios`, or similar.

```
fetch("https://example.com/api/profile")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

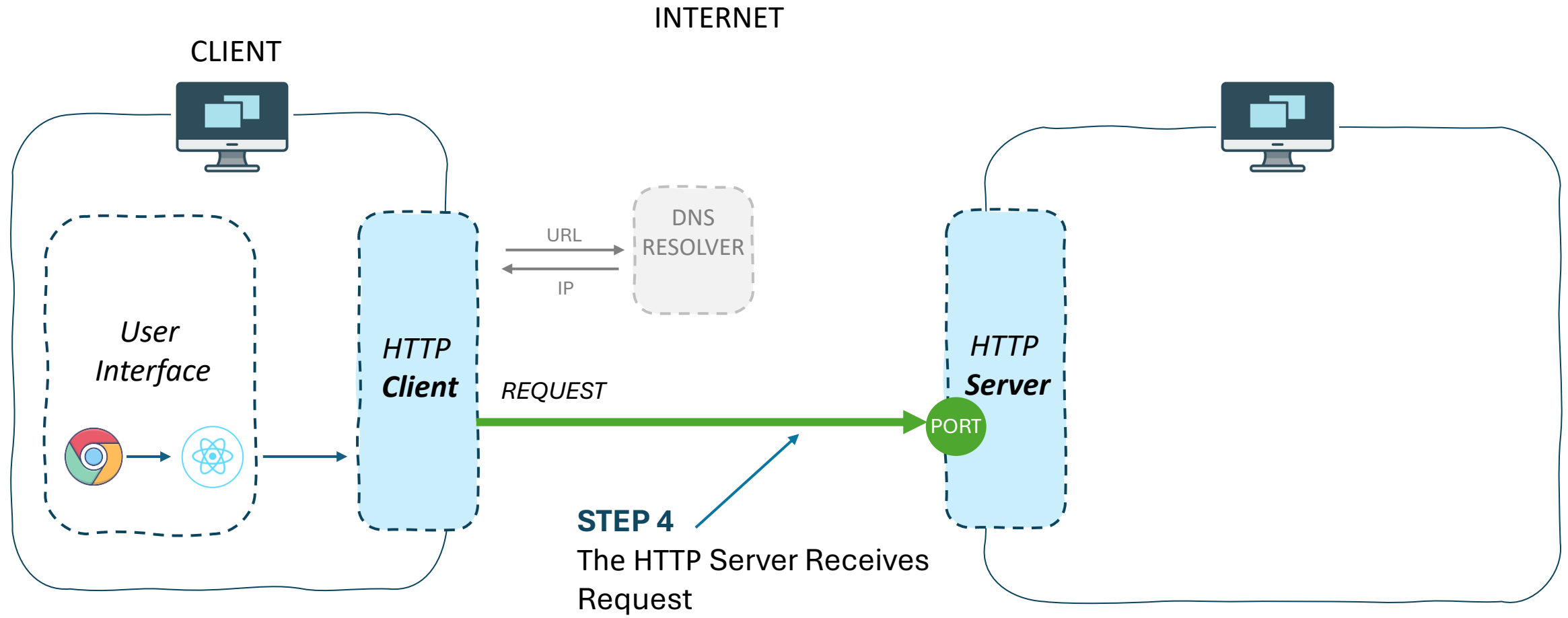
The Client–Server Flow



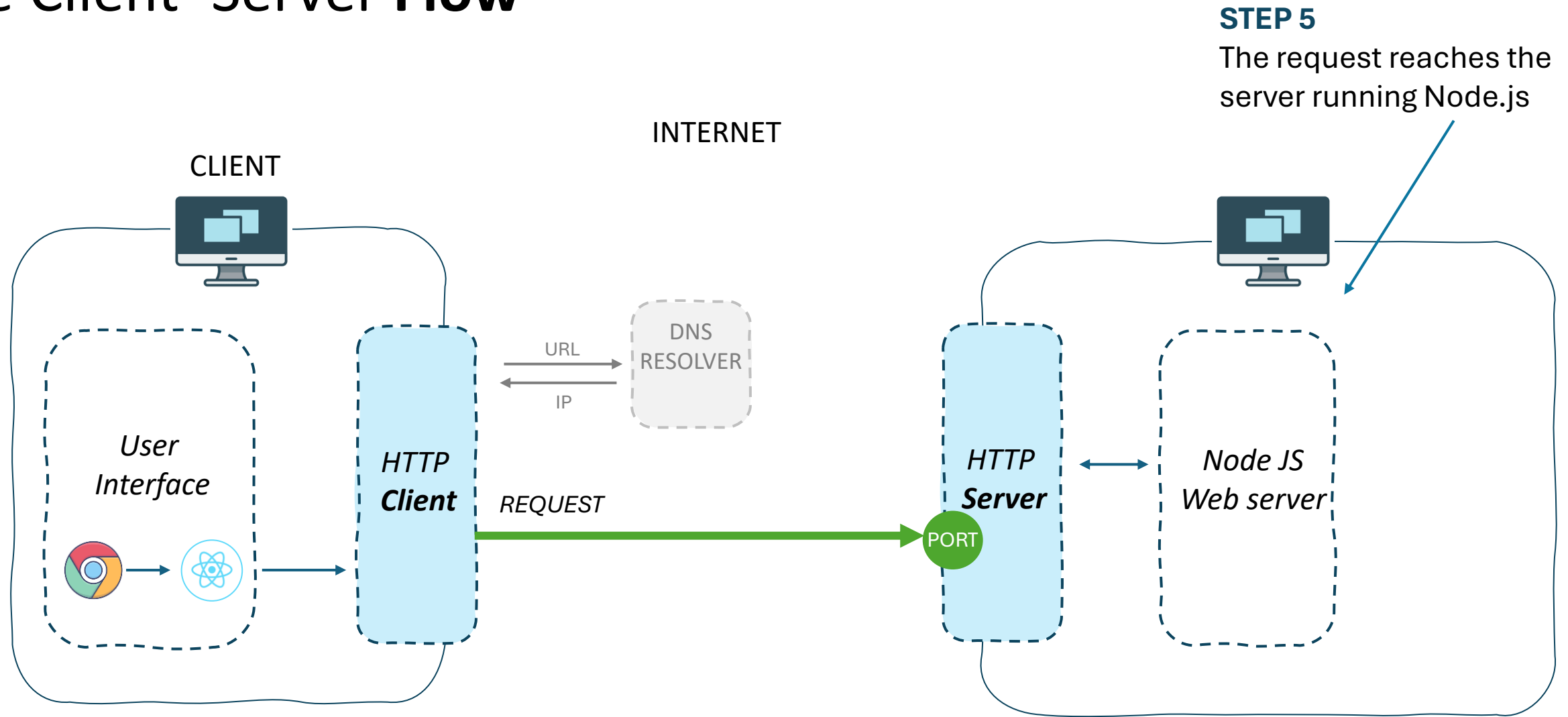
STEP 3

The browser checks the domain (example.com) and resolves it into an IP address.

The Client–Server Flow



The Client–Server Flow

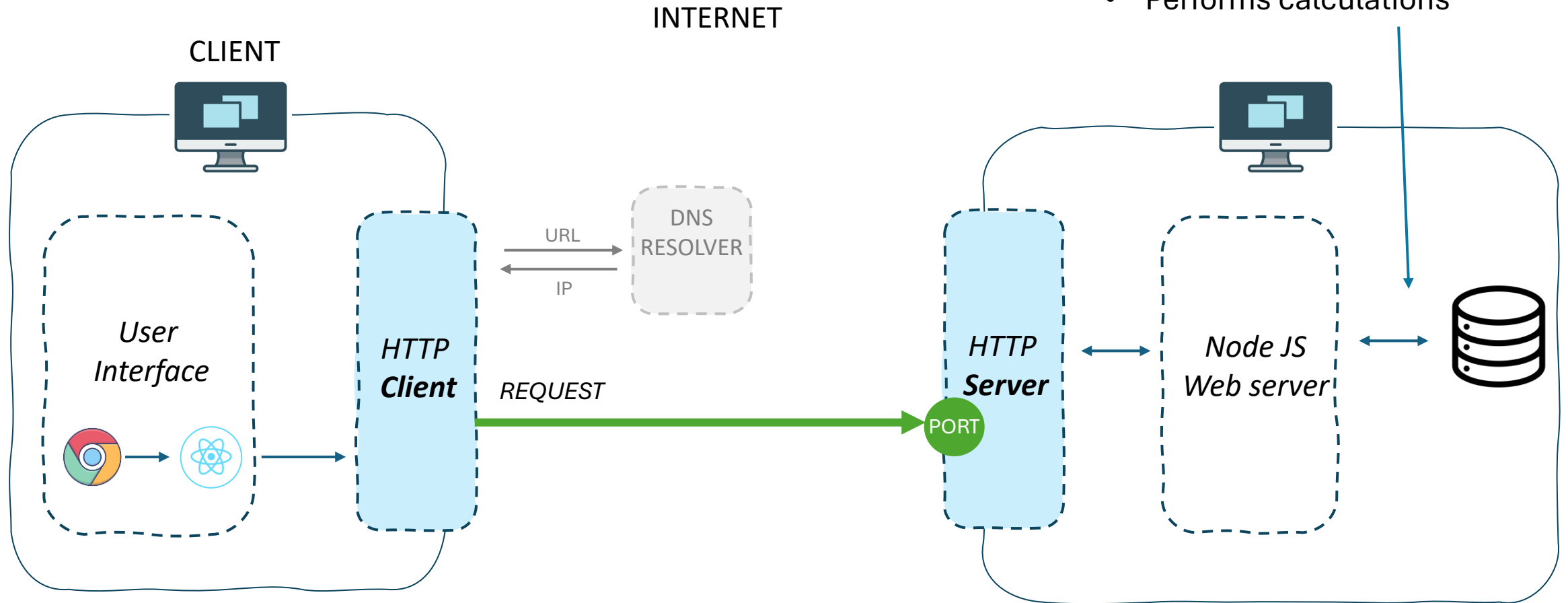


The Client–Server Flow

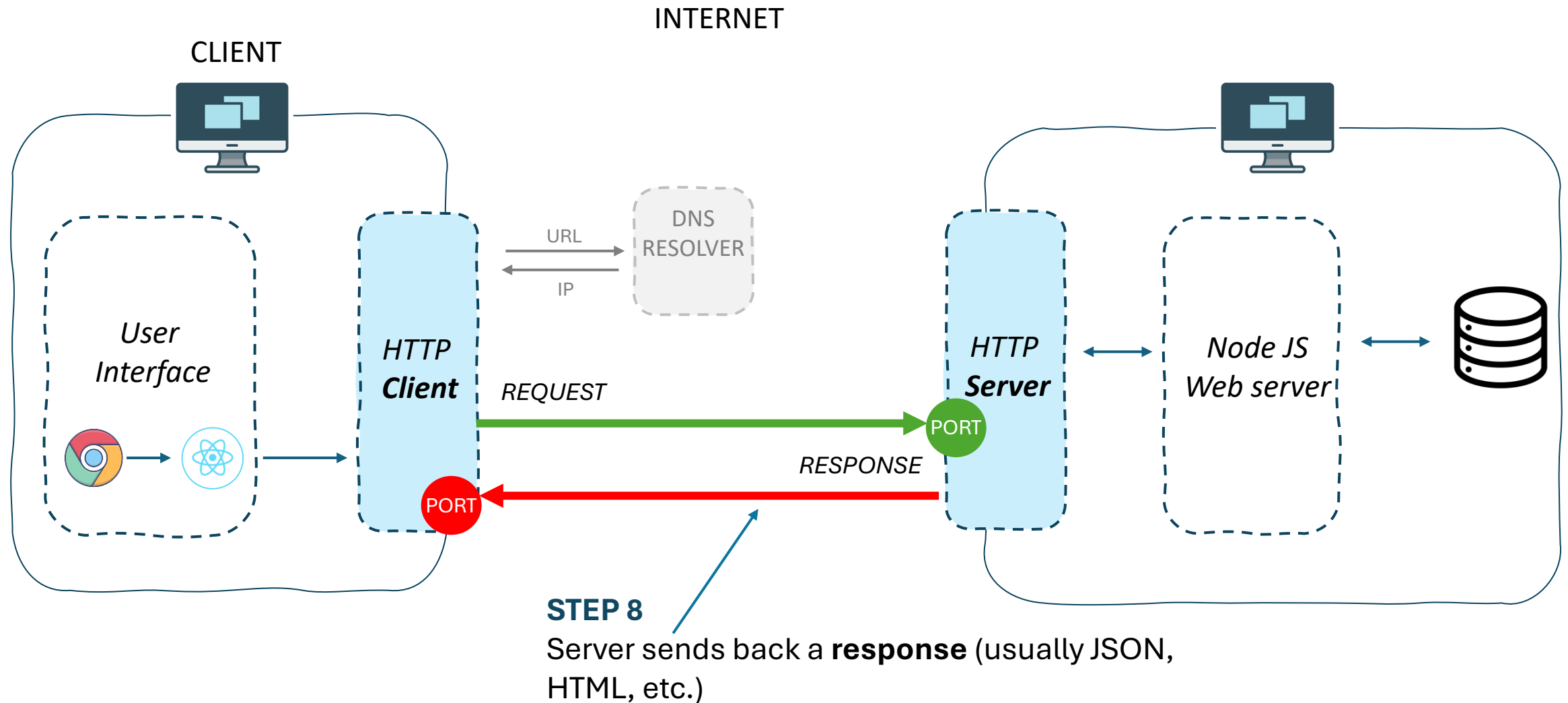
STEP 6

The server

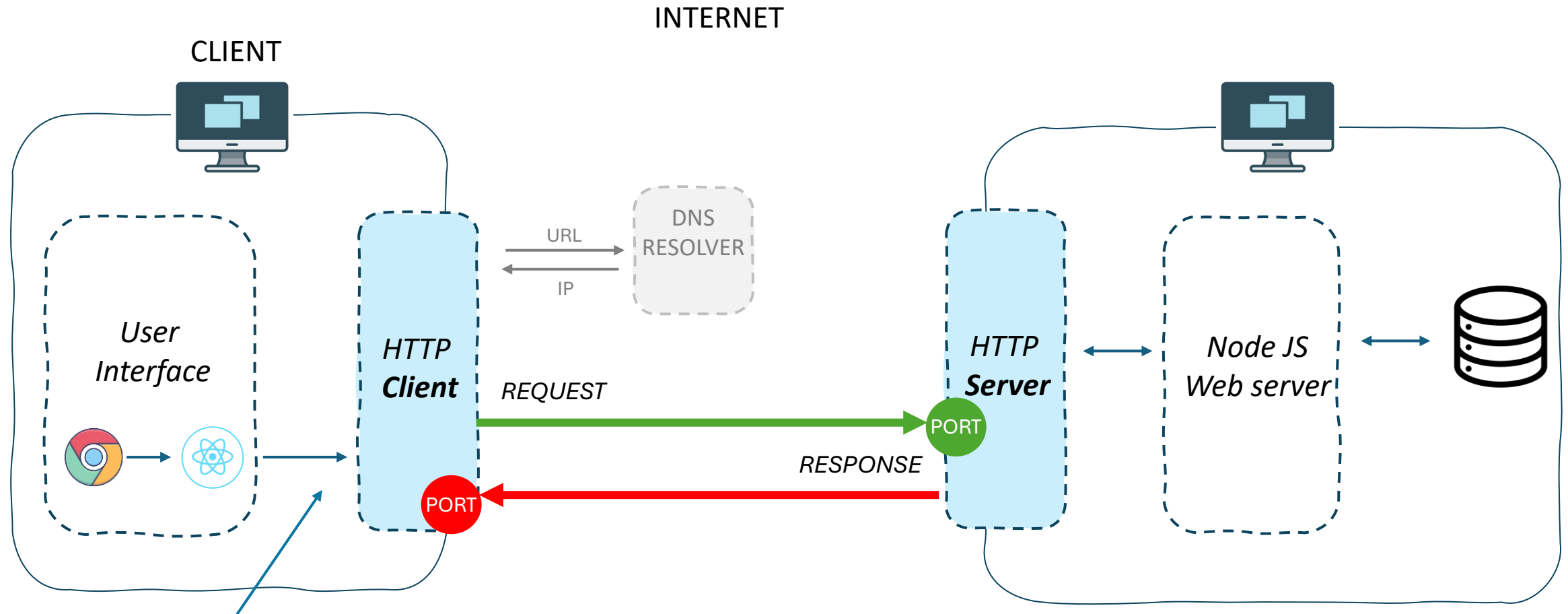
- Validates tokens
- Retrieves data from a **database**
- Performs calculations



The Client–Server Flow



The Client–Server Flow

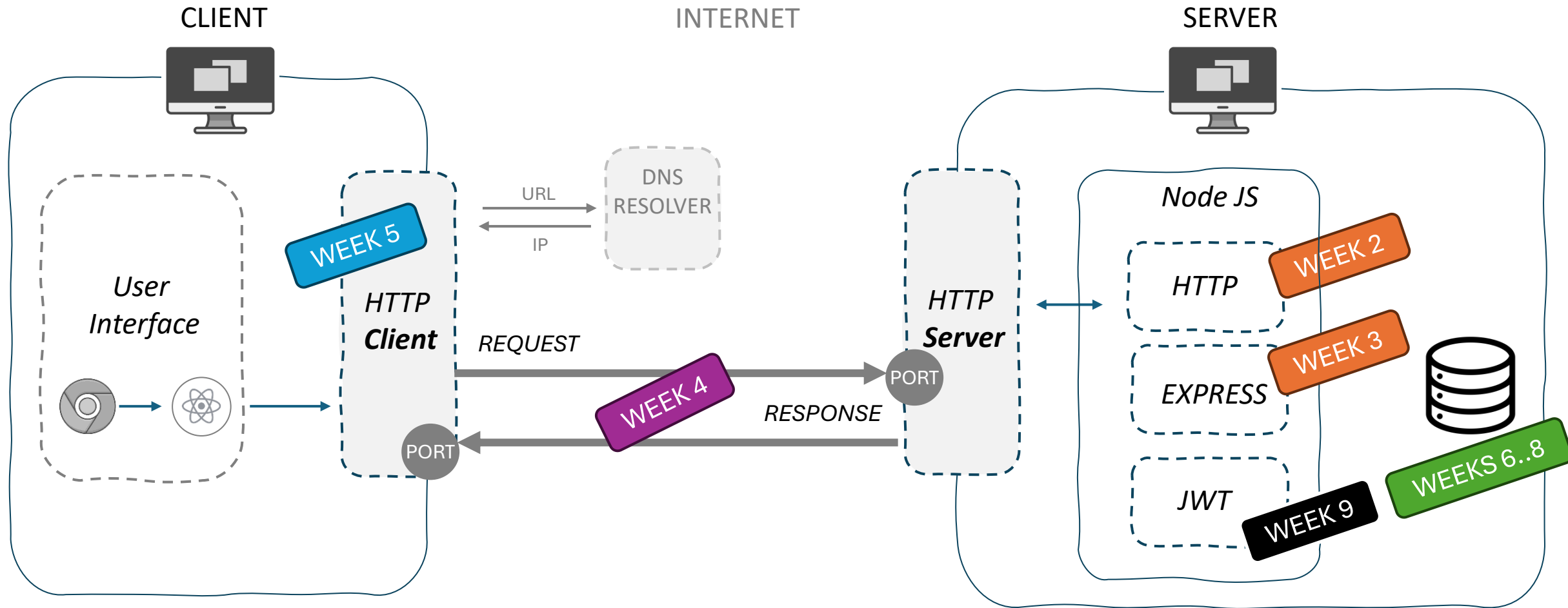


STEP 9

Browser Receives Response

Web app dynamically updates the UI

Overview of the course plan



What are the responsibilities of a server ?

Route Requests

Match the incoming request to the correct handler or function (e.g., /login, /api/users)

Interact with a Database

Read from and write to a database to fetch or update application data

Handle Client Requests

Receive and interpret HTTP requests from browsers or other clients

Send Responses Back to Clients

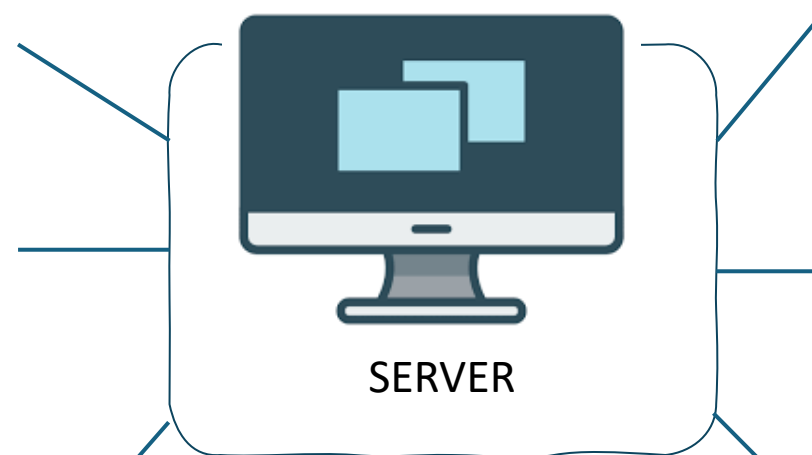
Return data (like HTML or JSON), status codes (e.g., 200, 404), and headers

Run Business Logic

Process data, perform calculations, enforce rules

Manage User Authentication & Authorization

Check user identity and control access to protected resources

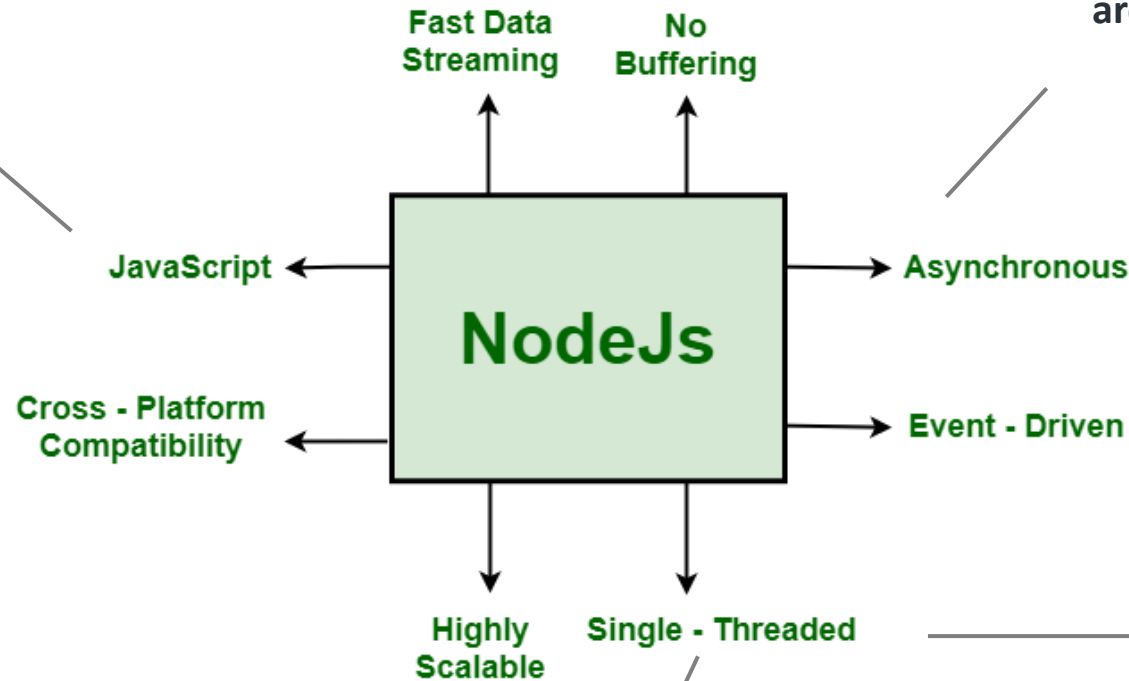


What is Node.JS ?

Node.js is a platform for **running JavaScript outside a browser**.

Node.js is built on the **V8 JavaScript engine**, which is known for its high performance

Node JS uses an **asynchronous, non-blocking architecture**, allowing multiple operations to execute concurrently



Node.js can run on **various operating systems** such as Windows, macOS, and Linux.

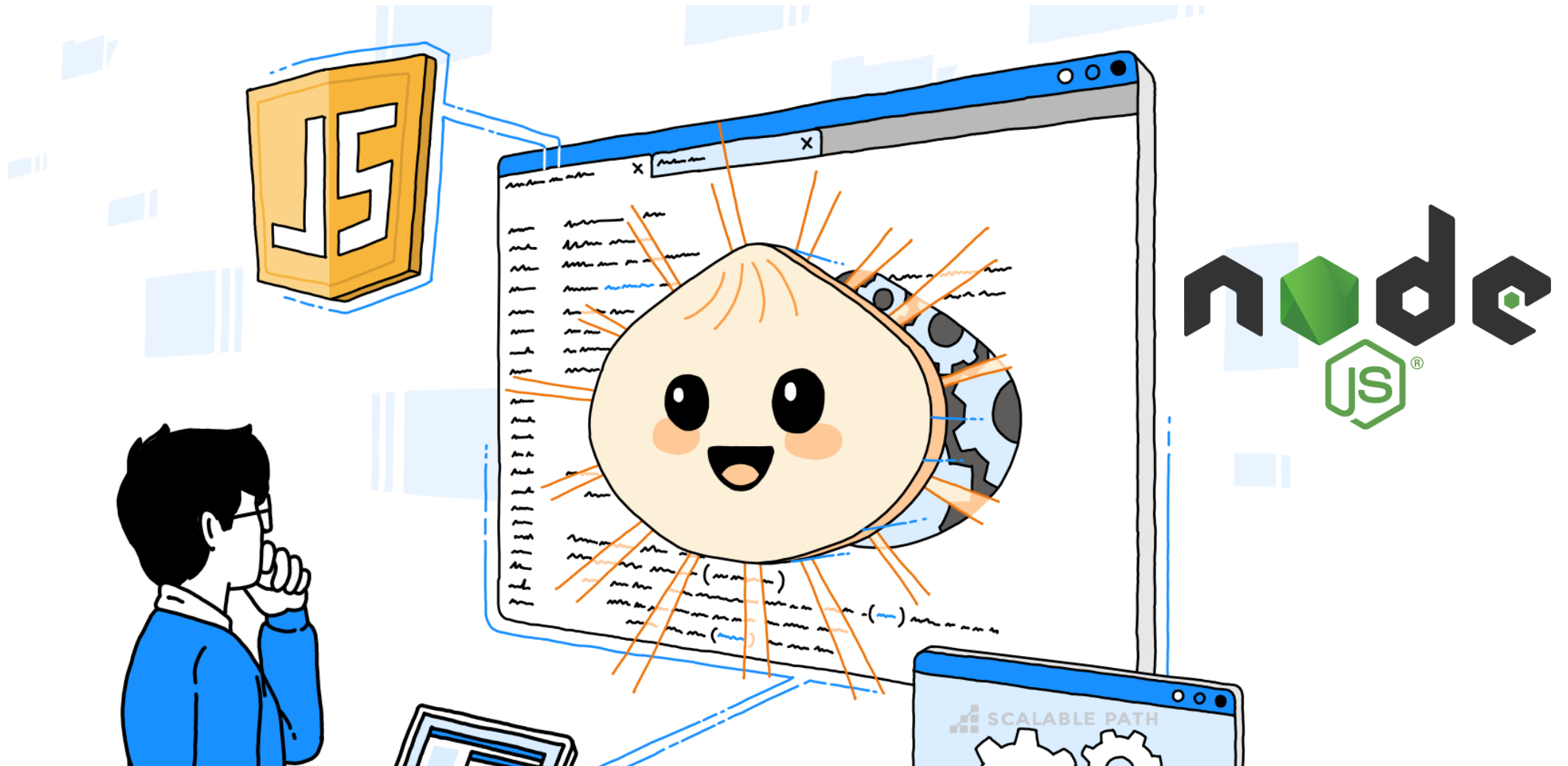
The server **operates on events**, responding to user actions or other **triggers**

Node.js uses a **single thread** to handle all requests

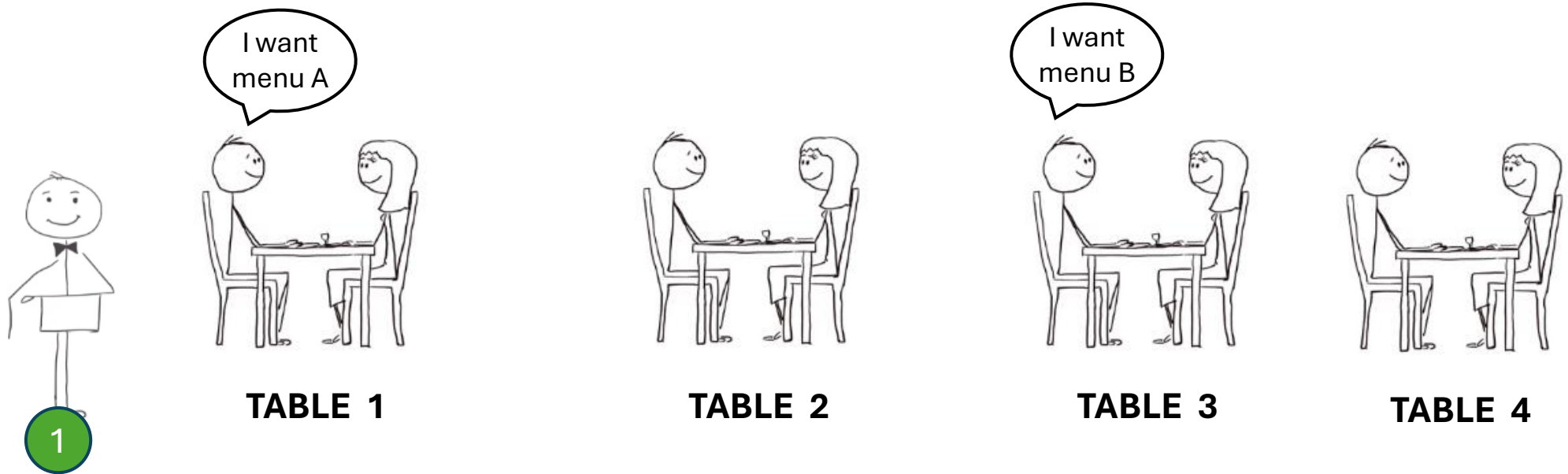
The **event loop** allows Node.js to handle thousands of connections **simultaneously**



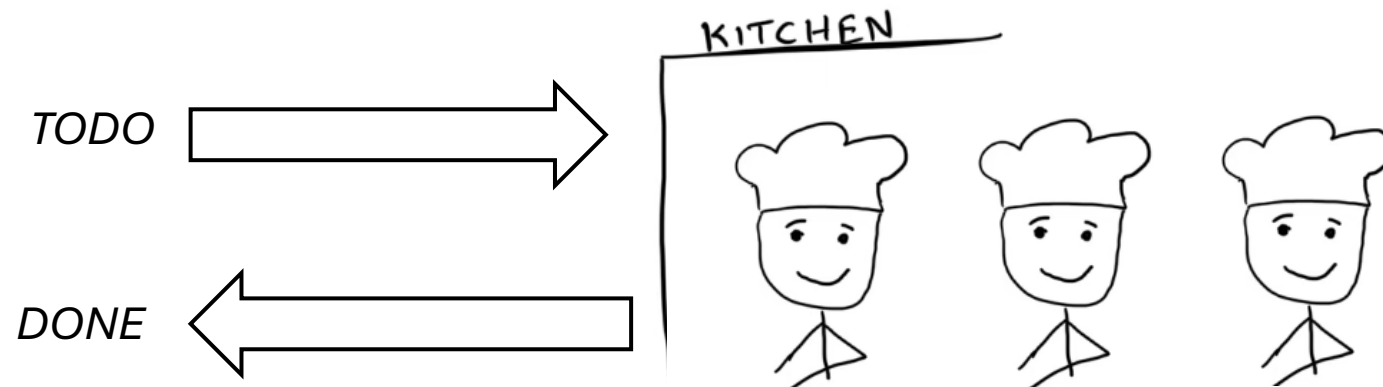
What makes Node.js fast ?



Let's first understand how restaurants work



The **waiter**
takes the order



Let's first understand how restaurants work



2

The **waiter** put the order in **TODO** list

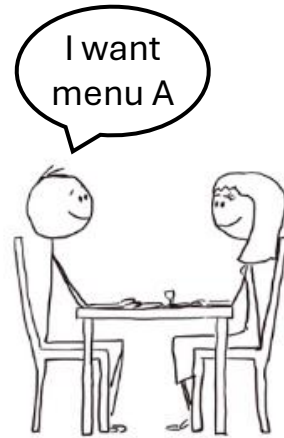


TABLE 1



TABLE 2

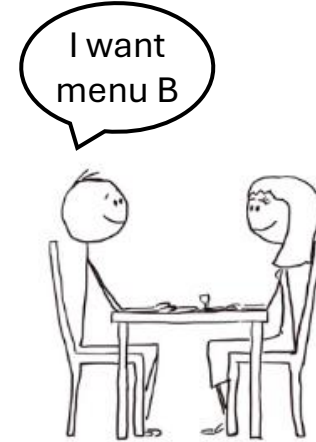
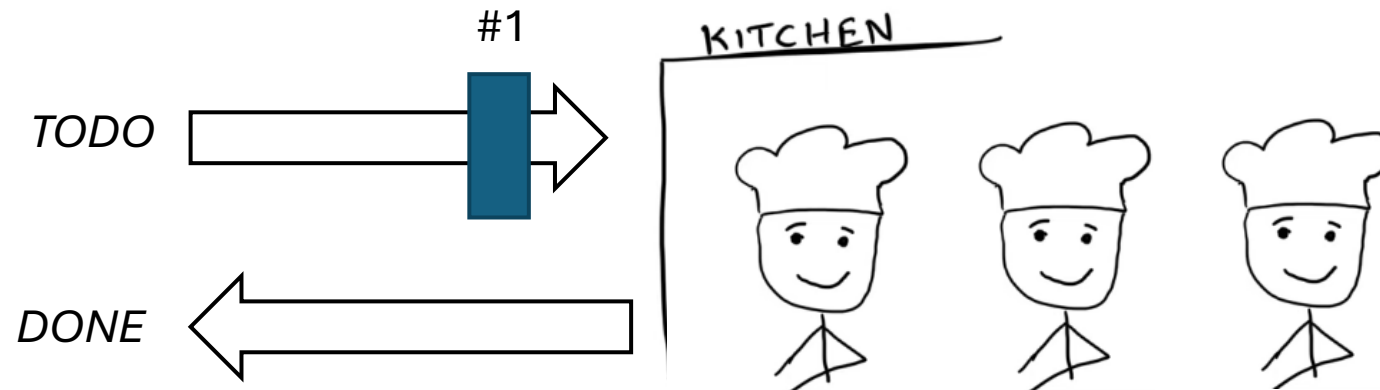


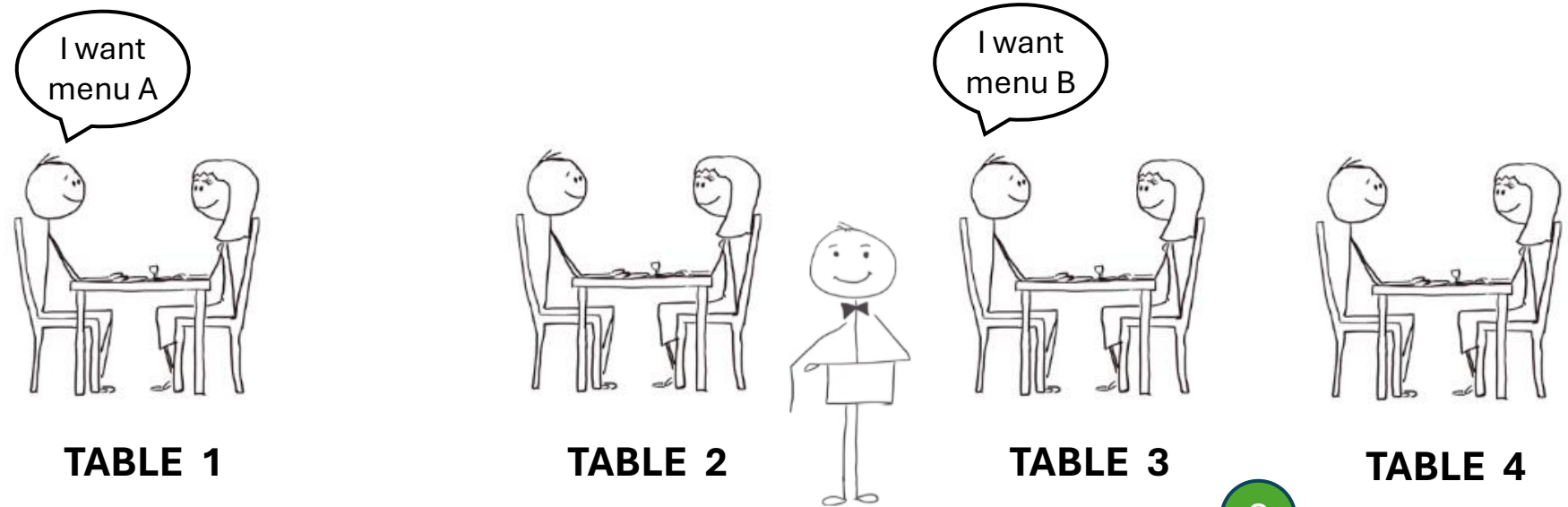
TABLE 3



TABLE 4

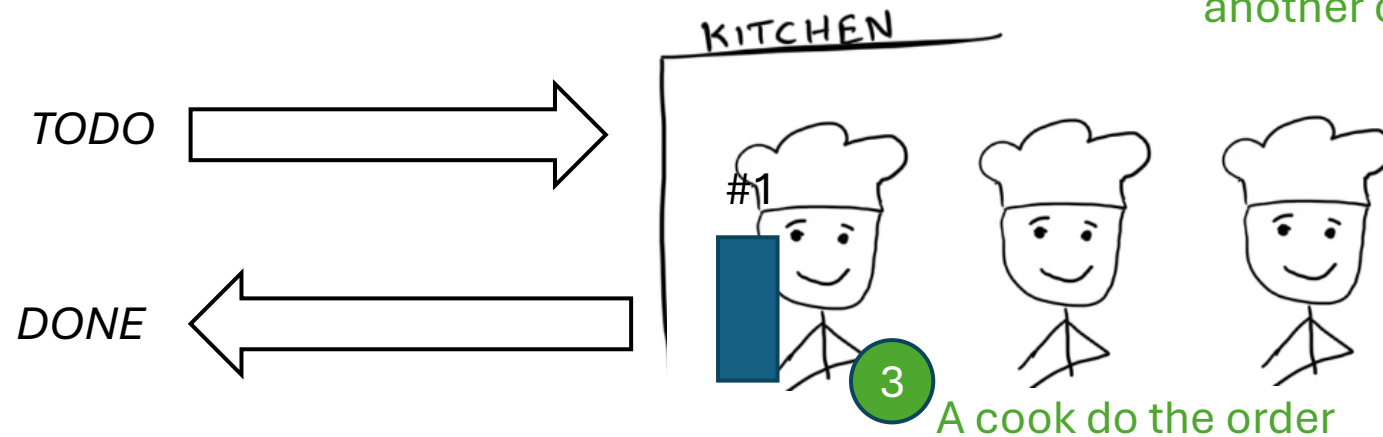


Let's first understand how restaurants work

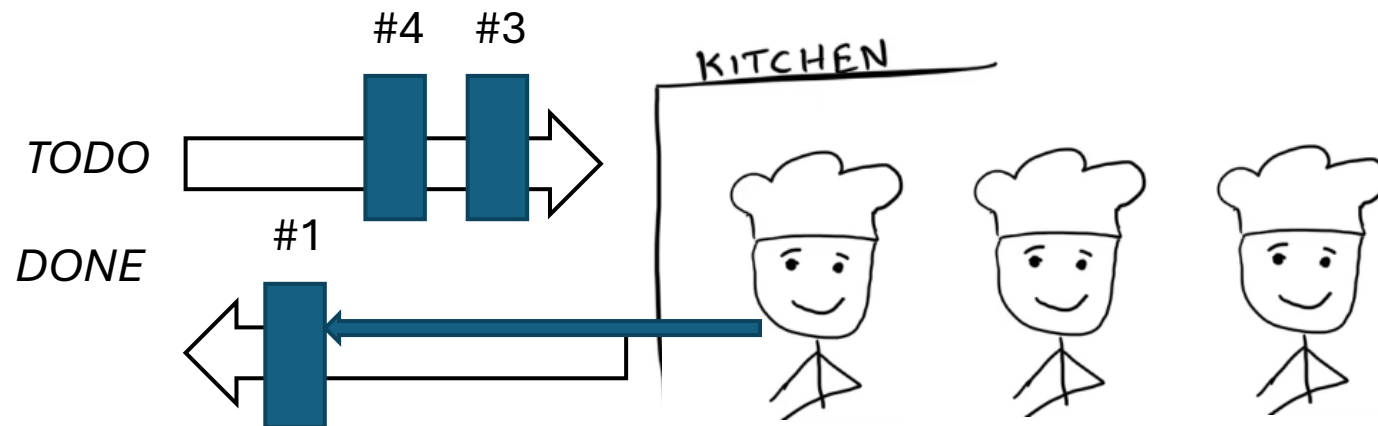
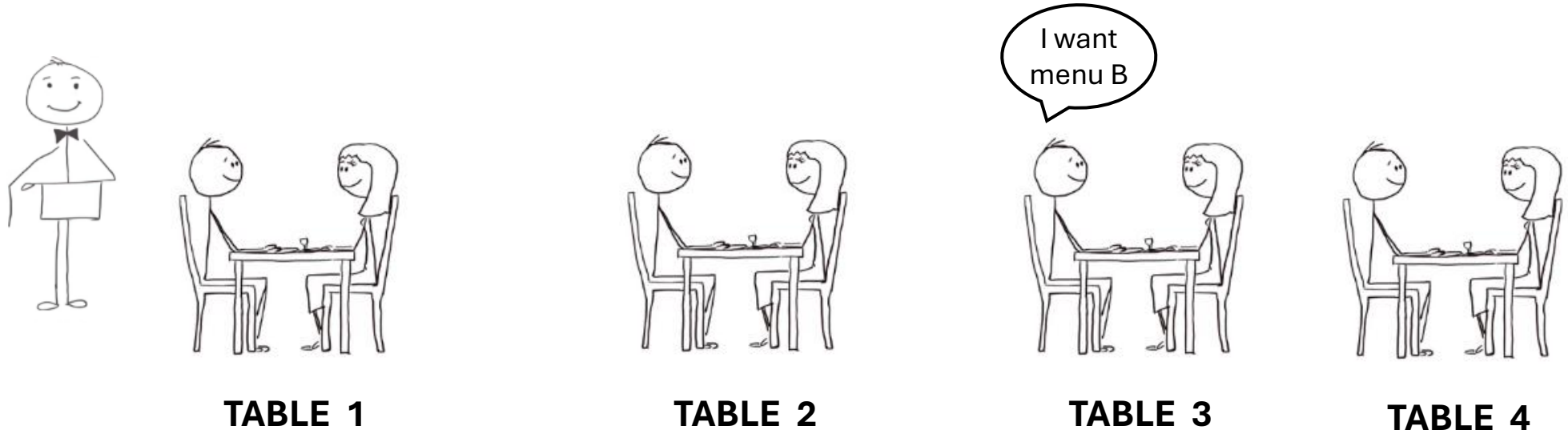


3

The waiter takes
another command



Let's first understand how restaurants work



4

The cook put the meal
on **DONE** list
And notify the waiter

Let's first understand how restaurants work



TABLE 1



TABLE 2

I want
menu B

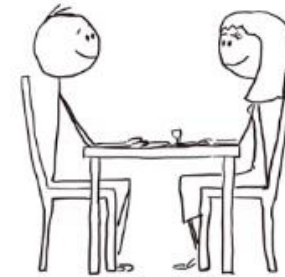


TABLE 3

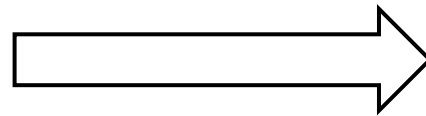


TABLE 4



#1

TODO



KITCHEN



#4



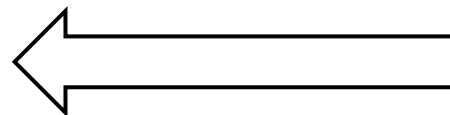
#3

The **cooks** are
cooking on different
orders

5

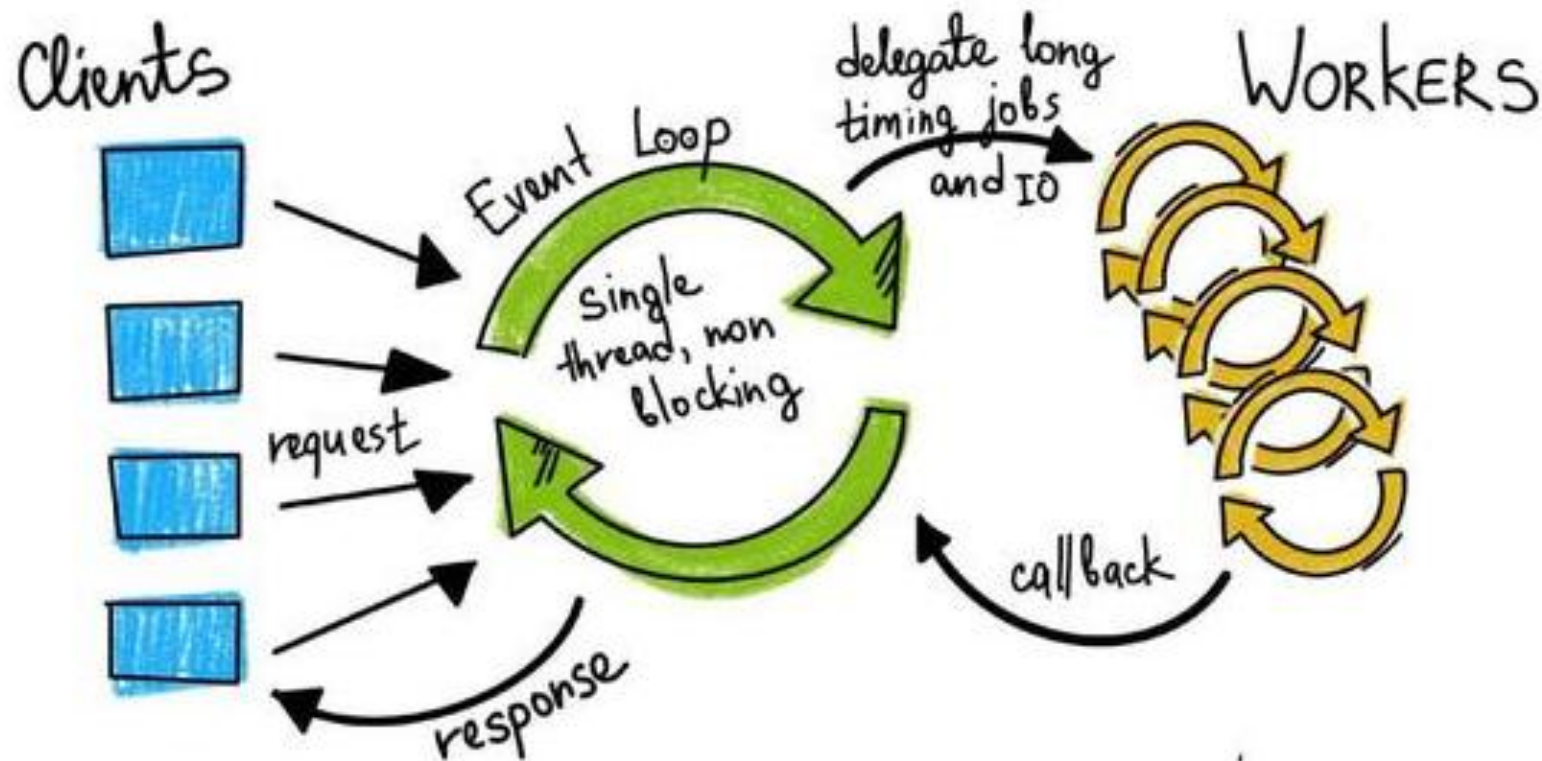
The **waiter** brings
The meal

DONE



What is an Event Loop?

The **Event Loop** in Node.js is a **continuous cycle** that checks the call stack and processes callbacks from the task queue, ensuring **non-blocking asynchronous operations**.

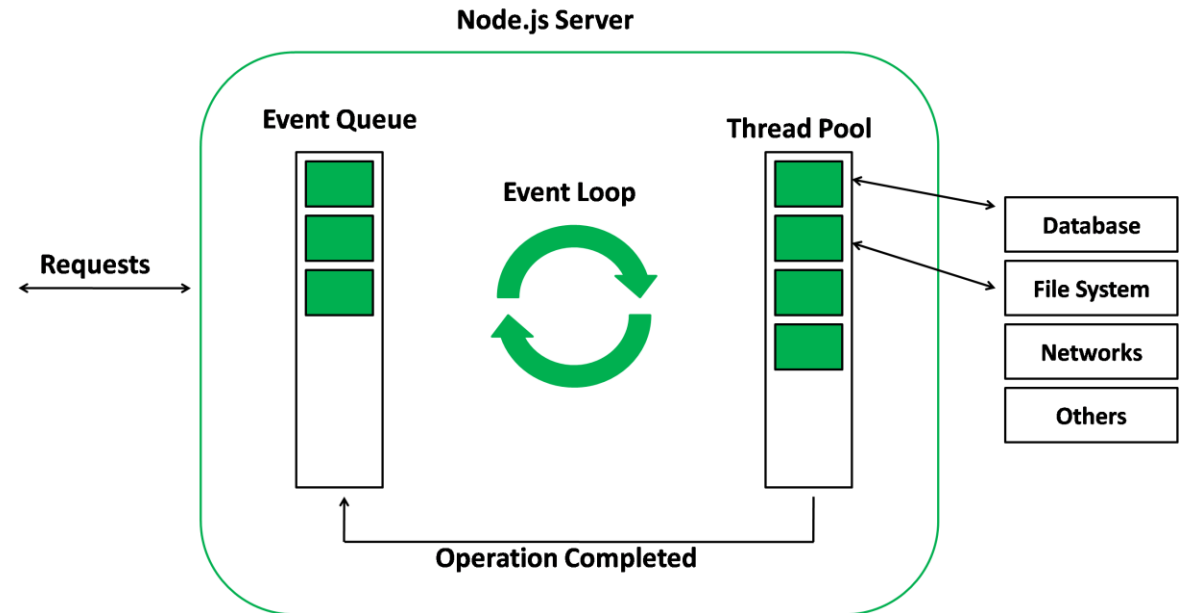


How comes Node.js is fast ?

*Instead of stopping everything to wait for slow operations — like reading a file or accessing a database - Node.js uses an **asynchronous approach**.*

When an operation is started, Node.js **doesn't block** the thread.

- It **registers a callback** to be triggered later and immediately moves on to handle other work.
- The **Event Loop** keeps checking for completed operations and runs the corresponding callbacks when they're ready.

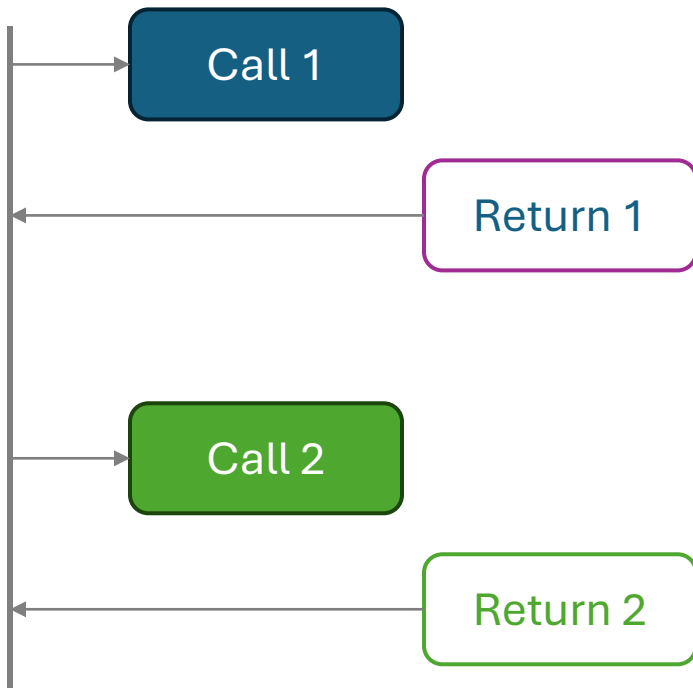


Synchronous VS Asynchronous

MORE INFO

SYNCHRONOUS

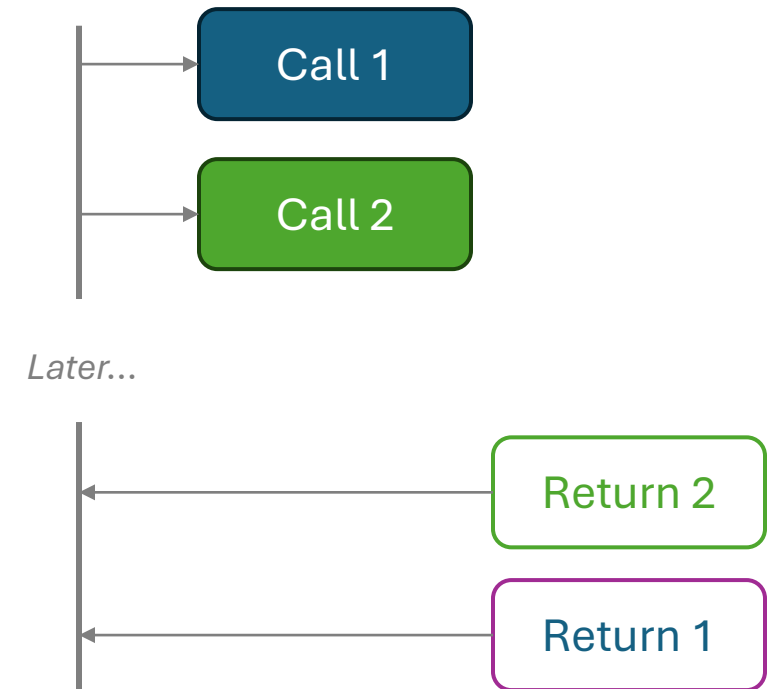
Code is executed sequentially, and **each operation must wait** for the previous one to complete.



ASYNCHRONOUS

Code **doesn't wait** for the completion of each operation and continues with the next one.

We use Callbacks & Futures



Let's call some **asynchronous** code

[LEARN MORE](#)

With **setTimeout**, we specify a **callback function** to execute **later**

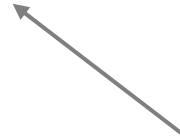
We will execute this function
After 500ms



```
function doSomething() {  
  // code  
}  
  
setTimeout(doSomething, 500);
```



A callback is just a **reference**
To a function, that will be called when needed

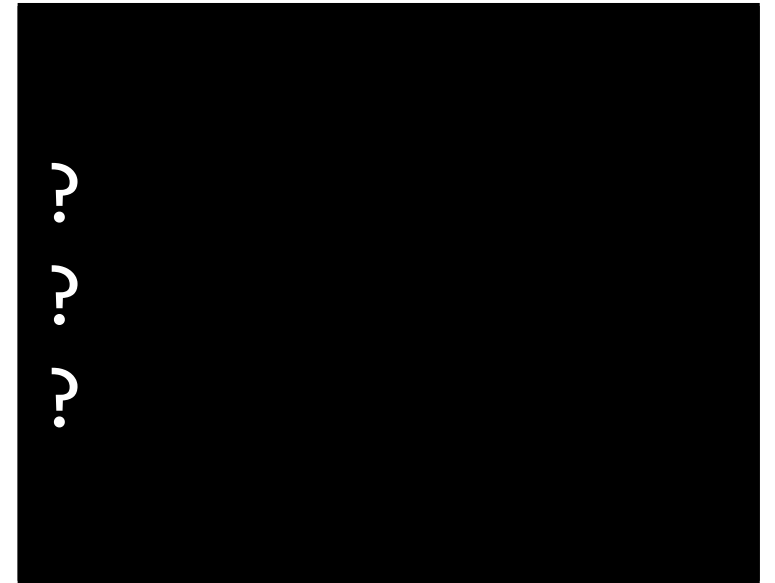


The callback will be called
After 500 ms

Synchronous VS Asynchronous

What will this code print out ?

```
function first() {  
  console.log(" 1 ");  
}  
  
function second() {  
  console.log("2 ");  
}  
  
function third(color) {  
  console.log("3 ", color);  
}  
  
first();  
second();  
third("orange");
```



ANSWER

Synchronous VS Asynchronous

What will this code print out ?

```
function first() {  
  console.log(" 1 ");  
}  
  
function second() {  
  console.log("2 ");  
}  
  
function third(color) {  
  console.log("3 ", color);  
}  
  
first();  
second();  
third("orange");
```

1
2
3 orange

Code here is
Executed synchronously

Synchronous VS Asynchronous

What will this code print out ?

```
setTimeout( () => { console.log("BANANA"); }, 3000);  
  
console.log("MANGO");  
  
setTimeout( () => { console.log("APPLE"); }, 2000);  
  
console.log("JACKFRUIT");  
  
setTimeout( () => { console.log("ORANGE"); }, 1000);
```

?.?
?.?
?.?

ANSWER

Synchronous VS Asynchronous execution

What will this code print out ?

```
setTimeout( () => { console.log("BANANA"); }, 3000);  
  
console.log("MANGO");  
  
setTimeout( () => { console.log("APPLE"); }, 2000);  
  
console.log("JACKFRUIT");  
  
setTimeout( () => { console.log("ORANGE"); }, 1000);
```

```
MANGO  
JACKFRUIT  
ORANGE  
APPLE  
BANANA
```

ANSWER

Understand the **asynchronous execution**

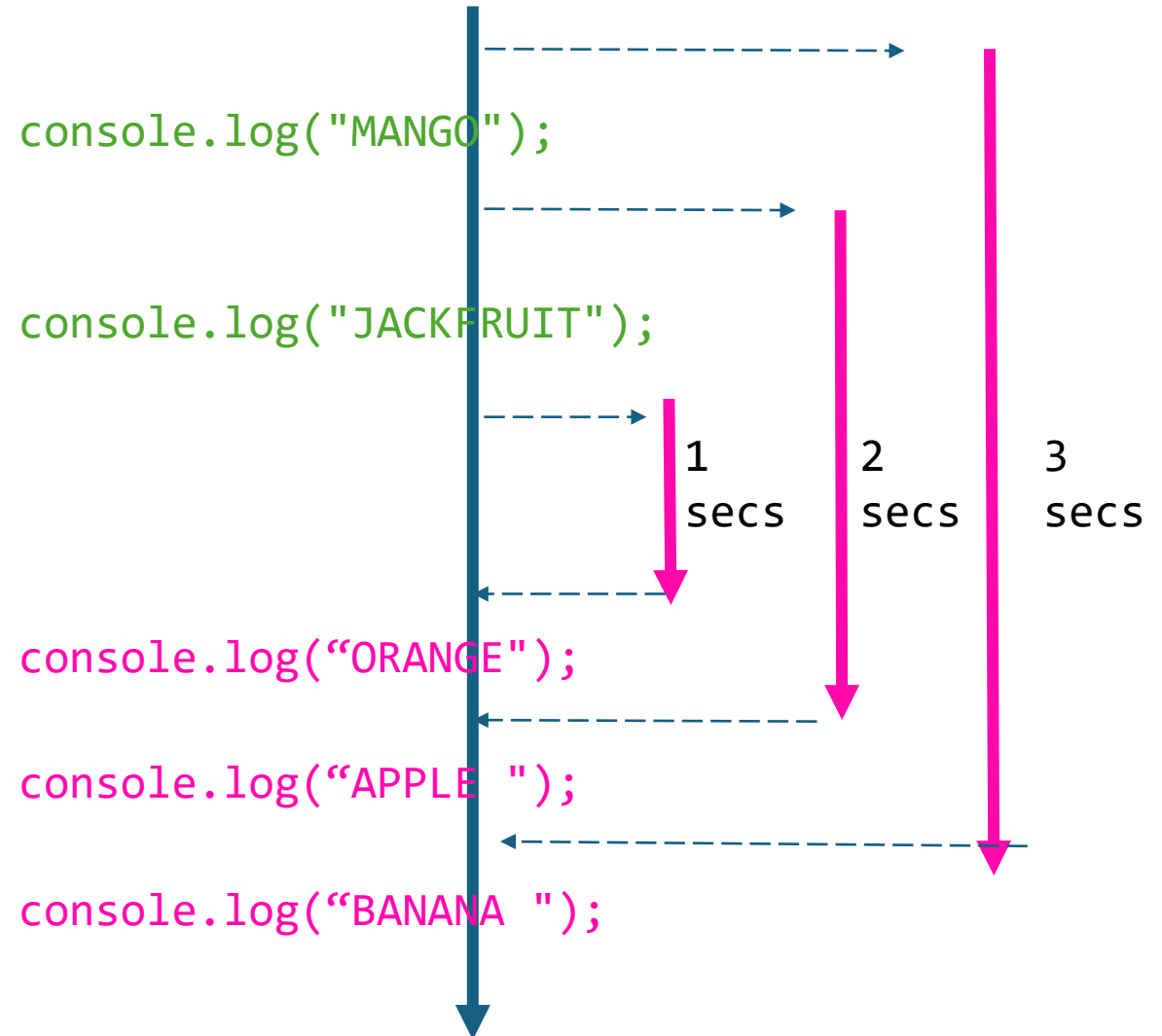
```
setTimeout( () =>
{ console.log("BANANA"); }, 3000);

console.log("MANGO");

setTimeout( () =>
{ console.log("APPLE"); }, 2000);

console.log("JACKFRUIT");

setTimeout( () =>
{ console.log("ORANGE"); }, 1000);
```





WHAT WE HAVE LEARNT



- ✓ Understand the flow of a **client-server architecture**
- ✓ Describe the core **responsibilities of a server**
- ✓ Explain what **Node.js** is and why it is used
- ✓ Illustrate how the Node.js **event loop works**
- ✓ Differentiate between **synchronous and asynchronous** programming



TO READ FOR THIS WEEK

CLIENT SERVER

- ✓ Client-Server Architecture – System Design [Read here](#)
- ✓ Client-Server Architecture Explained [Watch here](#)

NODE JS

- ✓ Node.js Crash Course [Watch here](#)
- ✓ Understanding the Node.js Event Loop [Watch here](#)
- ✓ What is Node.js [Read here](#)
- ✓ The Node.js Event Loop [Read here](#)

